



Programming Java

Multithreaded Programming

Incheon Paik



Contents

- An Overview of Threads
- Creating Threads
- Synchronization
- Deadlock
- Thread Communication



An Overview of Threads

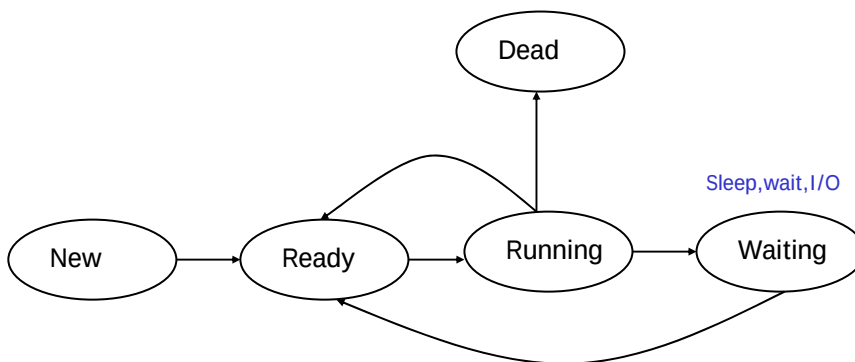
▪What is a Thread?

- A sequence of execution within a process
- A Lightweight process
- JVM manages and schedules threads
- Possible States:
(1) new (2) ready (3) running (4) waiting (5) dead



An Overview of Threads

▪Thread life cycle





Creating Threads

Extending Thread Class

```
class ThreadX extends Thread {
    public void run() {
        // process Threads
    }
}
```

Runnable Interface

```
class RunnableY implements Runnable {
    public void run() {
        // process Threads
    }
}
```

Starting a Thread

```
ThreadX tx = new ThreadX();
tx.start();
```

Starting a Thread

```
RunnableY ry = new RunnableY();
Thread ty = new Thread(ry);
ty.start();
```

run() method

```
public void run();
```

Thread Constructor

```
Thread()
Thread(Runnable r)
Thread(Runnable r, String s)
Thread(String s)
```



Methods in a Thread

Static Methods of the Thread Class

- *static Thread currentThread()*
- *static void sleep(long msec) throws InterruptedException*
- *static void sleep(long msec, int nsec) throws InterruptedException*
- *static void yield()*

Refer to the URL

<http://java.sun.com/j2se/1.4.2/docs/api/java/lang/Thread.html>

Instance Methods of Thread

- *String getName()*
- *int getPriority()*
- *boolean isAlive()*
- *void join() throws InterruptedException*
- *void join(long msec) throws InterruptedException*
- *void join(long msec, int nsec) throws InterruptedException*
- *void run()*
- *void setName(String s)*
- *void setPriority(int p)*
- *void start()*
- *String toString()*



Creating Threads

```
class ThreadX extends Thread {  
  
    public void run() {  
        try {  
            while(true) {  
                Thread.sleep(2000);  
                System.out.println("Hello");  
            }  
        } catch (InterruptedException ex) {  
            ex.printStackTrace();  
        }  
    }  
}  
  
class ThreadDemo1 {  
    public static void main(String args[]) {  
        ThreadX tx = new ThreadX();  
        tx.start();  
    }  
}
```

Result :
Hello
... 2 seconds
Hello

Body of run method



Creating Threads

```
class ThreadM extends Thread {  
    public void run() {  
        try {  
            for (int i = 0; i < 10; i++) {  
                Thread.sleep(1000);  
                System.out.println("ThreadM");  
            }  
        } catch (InterruptedException ex) {  
            ex.printStackTrace();  
        }  
    }  
}  
  
class ThreadN extends Thread {  
    public void run() {  
        try {  
            for (int i = 0; i < 20; i++) {  
                Thread.sleep(2000);  
                System.out.println("ThreadN");  
            }  
        } catch (InterruptedException ex) {  
            ex.printStackTrace();  
        }  
    }  
}
```

```
class JoinDemo1 {  
    public static void main(String args[]) {  
  
        ThreadM tm = new ThreadM();  
        tm.start();  
        ThreadN tn = new ThreadN();  
        tn.start();  
        try {  
            tm.join();  
            tn.join();  
            System.out.println("Both threads have  
finished");  
        } catch (Exception e) {  
            e.printStackTrace();  
        }  
    }  
}
```

join() method:
Waits for this thread to die.



Synchronization

Thread Scheduling

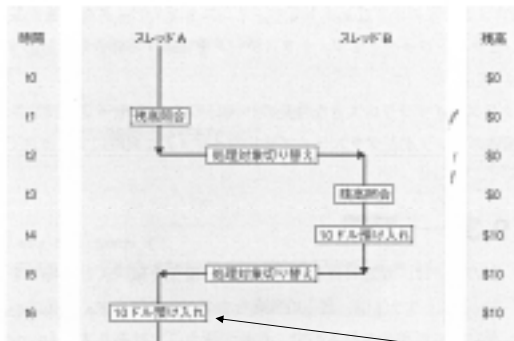


図 9-2 スレッドの遅れ

Synchronized Block

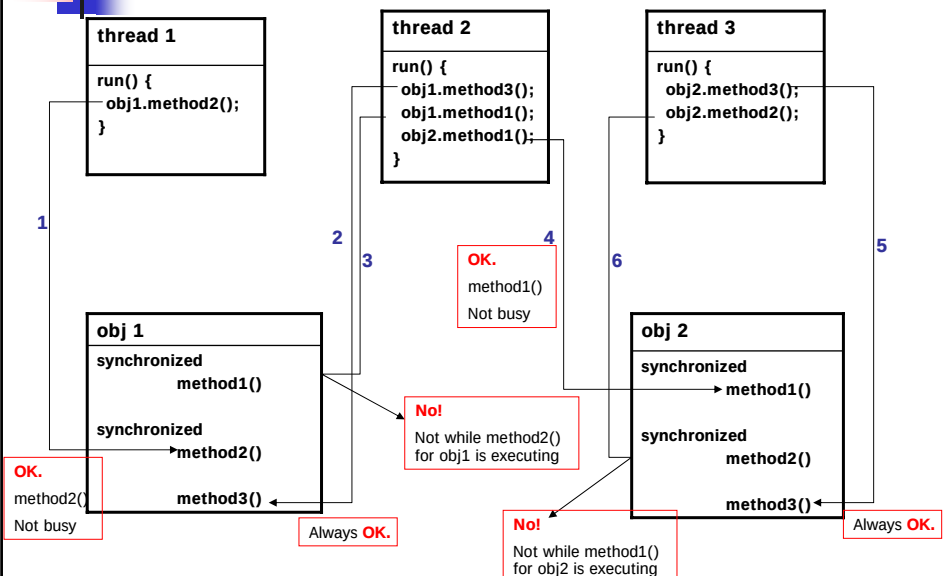
```
Synchronized (obj) {  
    // Process Block  
}
```

If a thread invokes a synchronized method, the object will be locked. If other threads invoke the synchronized method of that object, the threads will be blocked.

Data Corruption!
Need Synchronization



Locking Objects with Synchronized Methods





Synchronization

```
class Account {
    private int balance = 0;

    synchronized void deposit(int amount) {
    balance += amount;
}

    int getBalance() {
        return balance;
    }
}

class Customer extends Thread {
    Account account;

    Customer(Account account) {
        this.account = account;
    }

    public void run() {
        try {
            for (int i = 0; i < 100000; i++) {
                account.deposit(10);
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Result :
10,000,000

```
class BankDemo {
    private final static int NUMCUSTOMERS = 10;

    public static void main(String args[]) {

        // Create Account
        Account account = new Account();

        //Create and start customer threads
        Customer customers[] = new
        Customer[NUMCUSTOMERS];
        for (int i = 0; i < NUMCUSTOMERS; i++) {
            customers[i] = new Customer(account);
            customers[i].start();
        }

        //Wait for customer threads to complete
        for (int i = 0; i < NUMCUSTOMERS; i++) {
            try {
                customers[i].join();
            }
            catch (InterruptedException e) {
                e.printStackTrace();
            }
        }

        // Display Account balance
        System.out.println(account.getBalance());
    }
}
```

Java

Computer Industry Lab.



Deadlock

```
class A {
    B b;

    synchronized void a1() {
        System.out.println("Starting a1");
        b.b2();
    }

    synchronized void a2() {
        System.out.println("Starting a2");
    }
}

class B {
    A a;

    synchronized void b1() {
        System.out.println("Starting b1");
        a.a2();
    }

    synchronized void b2() {
        System.out.println("Starting b2");
    }
}

class Thread1 extends Thread {
    A a;

    Thread1(A a) {
        this.a = a;
    }
}
```

```
public void run() {
    for (int i = 0; i < 100000; i++)
        a.a1();
}

class Thread2 extends Thread {
    B b;

    Thread2(B b) {
        this.b = b;
    }

    public void run() {
        for (int i = 0; i < 100000; i++)
            b.b1();
    }
}

class DeadlockDemo {

    // Create Objects
    A a = new A();
    B b = new B();
    a.b = b;
    b.a = a;

    public static void main(String args[]) {
        // Create threads
        Thread1 t1 = new Thread1(a);
        Thread2 t2 = new Thread2(b);
        t1.start();
        t2.start();

        // Wait for threads to complete
        try {
            t1.join();
            t2.join();
        } catch (Exception e) {
            e.printStackTrace();
        }

        //Display Message
        System.out.println("Done!");
    }
}
```

```
// Create threads
Thread1 t1 = new Thread1(a);
Thread2 t2 = new Thread2(b);
t1.start();
t2.start();

// Wait for threads to complete
try {
    t1.join();
    t2.join();
} catch (Exception e) {
    e.printStackTrace();
}

//Display Message
System.out.println("Done!");
}
```

Condition for Deadlock:

- Mutual Exclusion
- Hold & Wait
- No-preemption
- Circular Wait

Result :
Starting a1
Starting b2
Starting a1
Starting b2
.....

Thread Communication

wait() Method

```
void wait() throws InterruptedException
void wait(long msec) throws InterruptedException
void wait(long msec, int nsec) throws InterruptedException
```

Not Runnable status

The wait() method allows a thread that is executing a synchronized method or statement block on that object to **release the lock** and wait for a **notification** from another thread.

Notify Method

```
void notify()
```

The notify() method allows a thread that is executing a synchronized method or statement block to notify another thread that is waiting for a lock on this object.

notifyAll() Method

```
void notifyAll()
```

Producer & Consumer Example

```
class Producer extends Thread {
    Queue queue;

    Producer(Queue queue) {
        this.queue = queue;
    }

    public void run() {
        int i = 0;
        while(true) {
            queue.add(i++);
        }
    }
}

class Consumer extends Thread {
    String str;
    Queue queue;

    Consumer(String str, Queue queue) {
        this.str = str;
        this.queue = queue;
    }

    public void run() {
        while(true) {
            System.out.println(str + ": " +
                queue.remove());
        }
    }
}
```

```
class Queue {
    private final static int SIZE = 10;
    int array[] = new int[SIZE];
    int r = 0;
    int w = 0;
    int count = 0;

    synchronized void add(int i) {
        while(count == SIZE) {
            try {
                wait();
            }
            catch(InterruptedException ie) {
                ie.printStackTrace();
                System.exit(0);
            }
        }
        array[w++] = i;
        if (w >= SIZE)
            w = 0;
        ++count;
        notifyAll();
    }

    synchronized int remove() {
        while(count == 0) {
            try {
                wait();
            }
            catch(InterruptedException ie) {
                ie.printStackTrace();
                System.exit(0);
            }
        }
    }
}
```

```
int element = array[r++];
if (r >= SIZE)
    r = 0;
--count;
notifyAll();
return element;
}

class ProducerConsumers {
    public static void main(String args[]) {
        Queue queue = new Queue();
        new Producer(queue).start();
        new Consumer("ConsumerA",
            queue).start();
        new Consumer("ConsumerB",
            queue).start();
        new Consumer("ConsumerC",
            queue).start();
    }
}
```



Exercise

- **Step 1 (Bank Example 1)**

Slide 9

You can use several ways to change the result of step 1.
As a way of those, I recommend you to use the synchronized statement block.

- **Synchronized statement block**

- We can specify a statement or a block of code in a program as synchronized.

“Form”

```
synchronized(theObject) {  
    // statement block  
}
```

No other statements or statements blocks in the program that are synchronized on the object can execute while this statement is executing.



Exercise

- **Step 2 (Bank Example 2)**

Slides 9 - 11

Remember the synchronized statement block and method

- **Step 3 (Producer and Consumer 1)**

Slides 9 - 14